R workshop part 1 - Introduction to R

Jan Gačnik

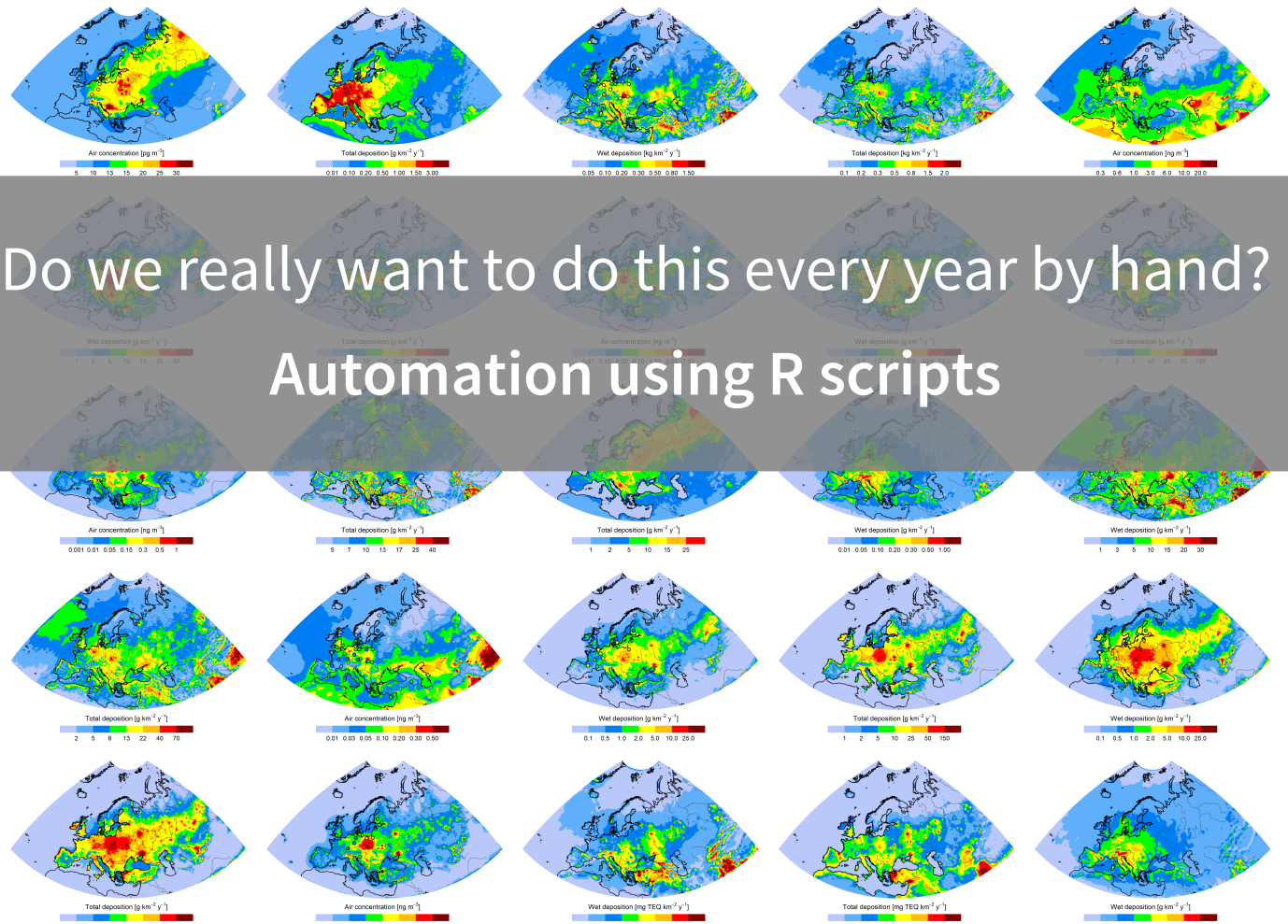
Prepared: 2026-10-26

Why R?

R is a programming language built for statistical computing

If one already knows Excel, Origin, SigmaPlot, etc. - why use R?

- *Free* to use
- *Large* community - support and packages
- Analysis and visualization are *easy to replicate*
- *Freedom* - almost anything is possible and modifiable
- *Automation* of plotting



Do we really want to do this every year by hand?

Automation using R scripts

Why R?

R vs python

Analogy: With what can you cook rice?



Very good rice can be cooked

…but also any other dish too



Perfect rice can be cooked

…but very few other dishes

Why R?

Why not just ChatGPT & Claude ?

- People who know R get dramatically more value from AI assistants than beginners
- AI as a copilot - you still need to know how to fly

Other considerations

- R script is a reproducible record of what was done with data. AI conversation is not.

> **ARIS and FAIR data**
>
> Open access to all research outputs from publicly funded projects - **data analysis and visualization included**

Workshop overview

1. Introduction to R
2. Data visualization
3. Data transformation
4. Data tidying
5. Projects and data import
6. Complete project demonstration

Workshop materials based on:

- Hadley Wickham, R for Data Science, https://r4ds.hadley.nz/
- Charles Lanfear, Introduction to R, https://clanfear.github.io/Intro_R_Workshop/

1) R and R studio orientation

RStudio

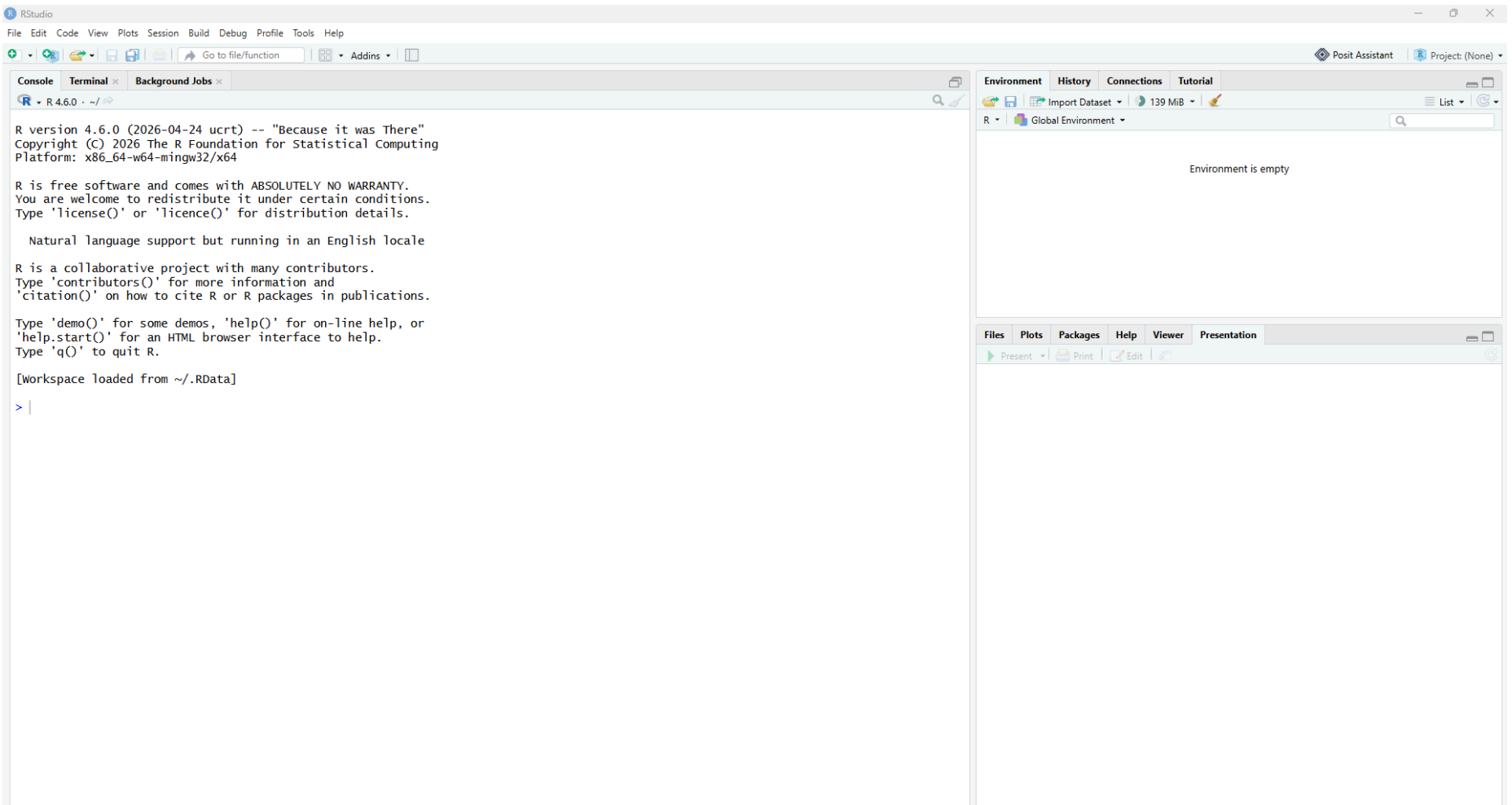
RStudio is a user-friendly application that gives a visual interface for writing, editing, and running R code.

RStudio can:

- Write & run R code with helpful features like auto-complete and syntax highlighting
- Enables viewing created objects (tables, graphs)
- Files, code, graphs, outputs → all in one window

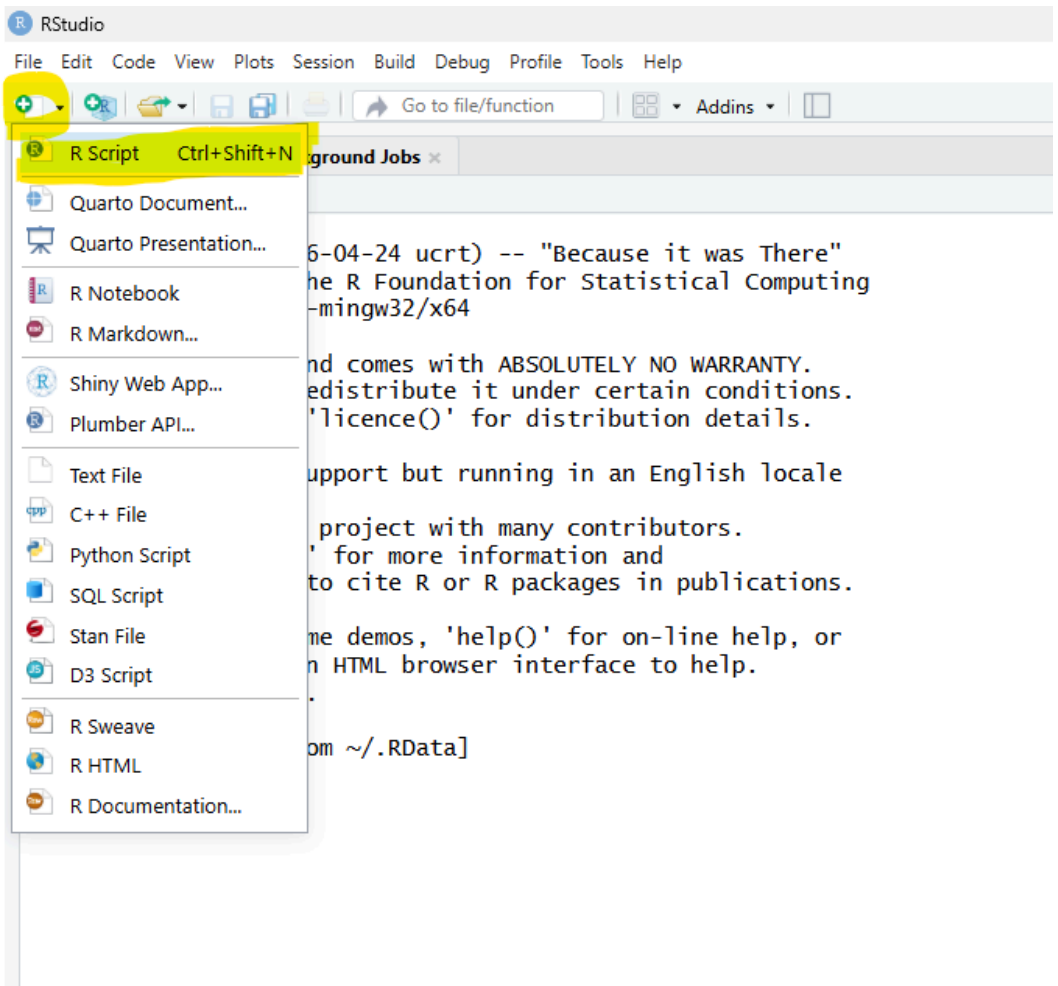
RStudio

This is how RStudio looks when opened for the first time:



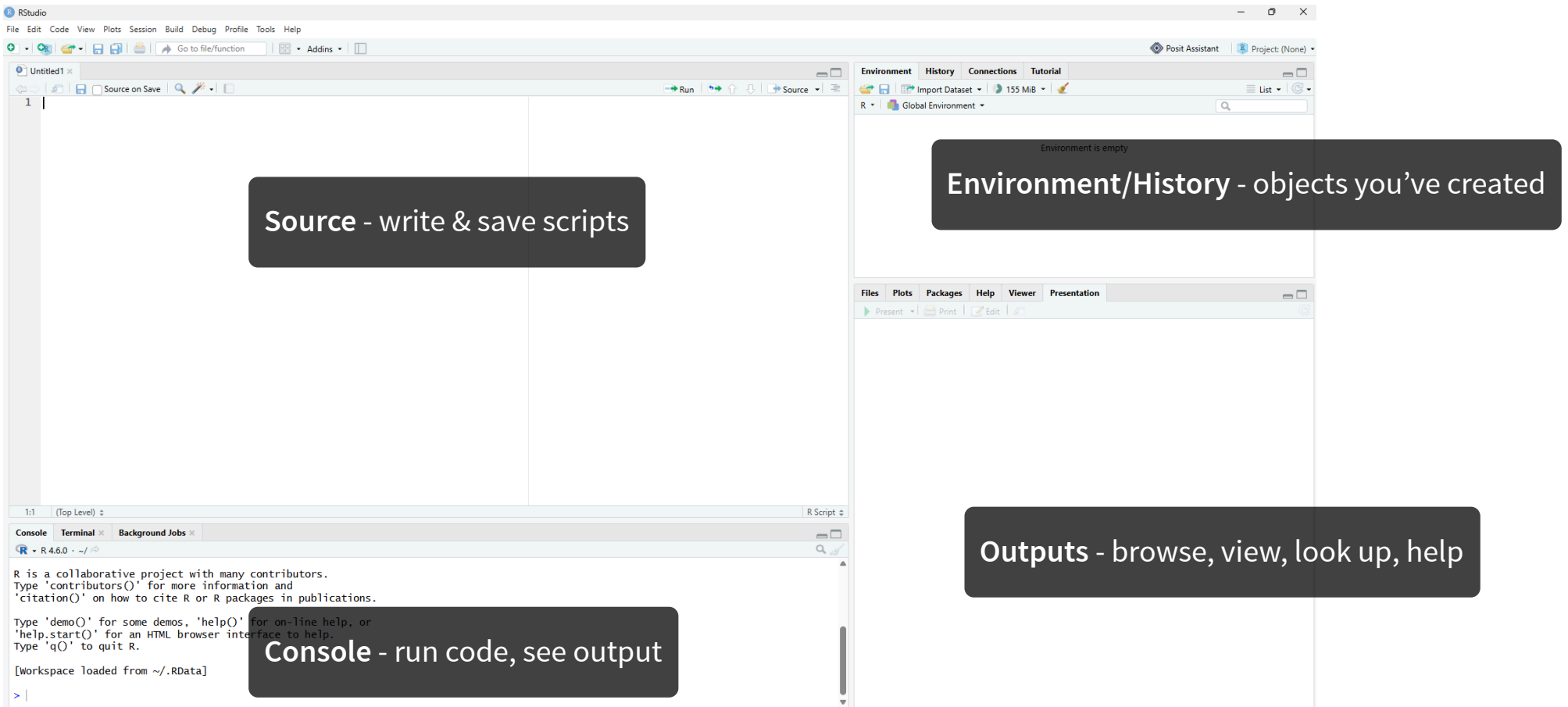
RStudio

We should open a new R script the following way:



RStudio

Finally, we have the 4 RStudio windows open, let's explain what is what



Editing and running code

Three ways to run R code in RStudio:

1. **Highlighting the lines in the editor window** and clicking *Run* at the top. Alternatively, `Ctrl` + `Enter` for Windows or ⌘ + `Enter` for Mac

2. **Clicking** anywhere on the code line (in the **editor window**) and pressing `Ctrl` + `Enter` or ⌘ + `Enter`

3. Typing or pasting the code directly into the **console window** and pressing `Enter`

Incomplete code

What if you mess up (e.g., forget to close brackets)?

- In console window, R might show `+` sign, indicating that the command has to be finished

```
> 12 * (2 + 6
+
```

- You can finish the command or hit `Esc` to get out of it

R as a calculator

In the console, type `12 + 34 + 56` and hit `Enter`.

- The outcome should look like this:

```
1  12 + 34 + 56
```

```
[1] 102
```

- The `[1]` indicates the numeric index of the first element in that line

In your editor window, try typing the line `sqrt(400)` and running it by `Ctrl` + `Enter` or ⌘ + `Enter`

```
1  sqrt(400)
```

```
[1] 20
```

Functions and help

`sqrt()` was an example of a **function** in R. To see more details about what each function does, type `?` before the function.

```
1 ?sqrt
```

Arguments are inputs that are passed to a function. In the case of `sqrt()`, the only argument is `x`, which can be a number, or a vector of numbers

Functions can get complex - documentation is your friend!

Comments in the code

By using the `#` sign, you can create a comment, which will be ignored when you run code.

- Comments are useful for describing what was done:

```
1 # This calculates square root of 400
2 sqrt(400)
```

```
[1] 20
```

- Comments are also useful to “comment-out” the code you want to ignore

```
1 # This will be skipped
2 # sqrt(400)
```

2) Creating and using objects

Creating and using objects

R stores *everything* as an object - data, functions, models, output. Objects are created using the assignment operator `<-`, shortcut for it: `Alt` + `-`

- Create an object, display it, and use it in calculations:

```
1 new_object <- 144
2 new_object
```

```
[1] 144
```

```
1 new_object + 10
```

```
[1] 154
```

```
1 sqrt(new_object)
```

```
[1] 12
```

Data types/classes

Every object in R has a **type** (also called a **class**)

- **numeric** - numbers, e.g. `144`, `3.14`
- **character** - text (strings), e.g. `"Atlantic"`
- **logical** - `TRUE` / `FALSE`
- **factor** - categorical data with fixed levels, e.g. `"low"`, `"medium"`, `"high"`

- You can check an object's type using `class()` function:

```
1 class(new_object)
```

```
[1] "numeric"
```

- Many errors in R happen because a function expected one type and got another

Creating vectors

A **vector** is a series of elements, such as numbers or characters

You can create a vector and store is as an object. Creating vectors is done using the function `c()` which stands for “combine” or “concatenate”:

```
1 numeric_vector <- c(4, 9, 16, 25, 36)
2 numeric_vector
```

```
[1]  4  9 16 25 36
```

- If you name an object the same name as an existing object *it will overwrite it*
- You can provide vectors as arguments to many functions:

```
1 sqrt(numeric_vector)
```

```
[1] 2 3 4 5 6
```

Creating character vectors

We often work with categorical data. A vector of text elements (called **strings** in programming jargon) is created by putting text in quotes:

```
1 string_vector <- c("Atlantic", "Pacific", "Arctic")
2 string_vector
```

```
[1] "Atlantic" "Pacific"  "Arctic"
```

- Functions can also be used on character vectors, such as the `sort()` function

```
1 sort(string_vector)
```

```
[1] "Arctic"   "Atlantic" "Pacific"
```

- As we go on, we will see functions which accept more than just one argument, such as the `decreasing = TRUE` argument of the `sort()` function:

```
1 sort(string_vector, decreasing = TRUE)
```

```
[1] "Pacific"  "Atlantic" "Arctic"
```

Logical vectors and comparisons

R can evaluate whether something is `TRUE` or `FALSE` using **comparison operators**:

- `==` equal to `!=` not equal to
- `>` / `<` greater/less than
- `>=` / `<=` greater/less than or equal to

```
1 numeric_vector > 20
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE
```

- The result is a **logical vector** - itself a type we can store, count, or use to filter data

Logical vectors and comparisons

R can evaluate whether something is `TRUE` or `FALSE` using **comparison operators**:

- `==` equal to `!=` not equal to
- `>` / `<` greater/less than
- `>=` / `<=` greater/less than or equal to

- `&` (and), `|` (or) combine multiple conditions:

```
1 numeric_vector > 10 & numeric_vector < 30
```

```
[1] FALSE FALSE  TRUE  TRUE FALSE
```

Missing values

Real datasets almost always have gaps. R represents a missing value as `NA`.

```
1 na_vector <- c(4, 9, NA, 25, 36)
2 na_vector
```

```
[1]  4  9 NA 25 36
```

- `NA` is “contagious” - most calculations involving `NA` return `NA`:

```
1 sum(na_vector)
```

```
[1] NA
```

Missing values

Real datasets almost always have gaps. R represents a missing value as `NA`.

```
1 na_vector <- c(4, 9, NA, 25, 36)
2 na_vector
```

```
[1]  4  9 NA 25 36
```

- Use `is.na()` to detect missing values, and `na.rm = TRUE` to ignore them in calculations:

```
1 is.na(na_vector)
```

```
[1] FALSE FALSE  TRUE FALSE FALSE
```

```
1 sum(na_vector, na.rm = TRUE)
```

```
[1] 74
```

- Keep an eye out for `NA`s - forgetting about them is one of the most common beginner mistakes!

More complex objects

Same principles shown with vectors can be used with more complex objects, such as **matrices**, **arrays**, **lists**, and **dataframes**

- **Dataframe** is the object we will focus on the most
  - The same concept as an Excel table, with more defined contents
  - Can store different vector types inside it, e.g. a character column, a numeric column, a logical column, …

More complex objects

As an example data frame, we will use the `penguins` dataset



- A built-in dataset included with recent versions of R
- Measurements from **344 adult penguins** representing **three species** on **three islands** near Palmer Station, Antarctica
- Eight variables describing species, island, bill size, flipper length, body mass, sex, and year

Loading the penguins dataset

Use `data()` to place the dataset in our environment, then use `head()` to inspect its first six rows:

```
1 data(penguins)
2 head(penguins)
```

```
  species    island bill_len bill_dep flipper_len body_mass    sex year
1  Adelie Torgersen     39.1     18.7         181      3750   male 2007
2  Adelie Torgersen     39.5     17.4         186      3800 female 2007
3  Adelie Torgersen     40.3     18.0         195      3250 female 2007
4  Adelie Torgersen       NA       NA          NA        NA   <NA> 2007
5  Adelie Torgersen     36.7     19.3         193      3450 female 2007
6  Adelie Torgersen     39.3     20.6         190      3650   male 2007
```

Accessing rows and columns

Data frames have two dimensions: **rows** and **columns**. Use `object[rows, columns]` to select part of a data frame.

- R starts counting at **1**. Leaving the row or column position empty selects **all** elements in that dimension.

```
1 penguins[2, ] # Second row, all columns
```

```
  species    island bill_len bill_dep flipper_len body_mass    sex year
2  Adelie Torgersen     39.5     17.4         186      3800 female 2007
```

```
1 penguins[1:3, 3:5] # Rows 1-3 and columns 3-5
```

```
  bill_len bill_dep flipper_len
1     39.1     18.7         181
2     39.5     17.4         186
3     40.3     18.0         195
```

Accessing columns

Columns can be selected by their **position index** or their **name**. When `[, ]` selects exactly one column from a base R data frame, the result is simplified to a vector.

```
1  penguins[, 3] # Third column, selected by POSITION
```

```
 [1] 39.1 39.5 40.3   NA 36.7 39.3 38.9 39.2 34.1 42.0 37.8 37.8 41.1 38.6 34.6
[16] 36.6 38.7 42.5 34.4 46.0 37.8 37.7 35.9 38.2 38.8 35.3 40.6 40.5 37.9 40.5
[31] 39.5 37.2 39.5 40.9 36.4 39.2 38.8 42.2 37.6 39.8 36.5 40.8 36.0 44.1 37.0
[46] 39.6 41.1 37.5 36.0 42.3 39.6 40.1 35.0 42.0 34.5 41.4 39.0 40.6 36.5 37.6
```

```
1  penguins[, "bill_len"] # The same column, selected by NAME
```

```
 [1] 39.1 39.5 40.3   NA 36.7 39.3 38.9 39.2 34.1 42.0 37.8 37.8 41.1 38.6 34.6
[16] 36.6 38.7 42.5 34.4 46.0 37.8 37.7 35.9 38.2 38.8 35.3 40.6 40.5 37.9 40.5
[31] 39.5 37.2 39.5 40.9 36.4 39.2 38.8 42.2 37.6 39.8 36.5 40.8 36.0 44.1 37.0
[46] 39.6 41.1 37.5 36.0 42.3 39.6 40.1 35.0 42.0 34.5 41.4 39.0 40.6 36.5 37.6
```

`[]` vs `[[]]`?

A data frame is also a collection of column vectors. The operator determines whether we keep the data-frame structure or extract the column itself:

- `penguins["bill_len"]` returns a **one-column data frame**.
- `penguins[["bill_len"]]` extracts the **numeric vector**.
- `penguins$bill_len` extracts the same vector using a convenient shortcut.

In short: `[]` keeps the **container**; `[[]]` and `$` take the values **out of the container**.

Accessing rows and columns

Command	What does it return?
`penguins[1:3, 3:5]`	A data frame containing rows 1-3 and columns 3-5
`penguins[, 3]`	The third column as a vector
`penguins[, "bill_len"]`	The named column as a vector
`penguins["bill_len"]`	A one-column data frame
`penguins[["bill_len"]]`	The named column as a vector
`penguins$bill_len`	The same vector, using a named shortcut

3) Packages

Installing and loading packages

Packages contain useful functions (and sometimes data) created by the community

- These are not included in base R
- The Comprehensive R Archive Network (CRAN): packages go through a *peer-review process by statisticians and computer scientists*

CRAN packages are installed using `install.packages()` - only once is needed

```
1 install.packages("tidyverse") # Run first
2 library("tidyverse") # Run second
```

Useful R packages

Too many to list, but `tidyverse` is by far the most useful

A package that includes a *family of packages*:

- `ggplot2` for *plotting*
- `dplyr` and `tidyr` for *data manipulation*
- `lubridate` for working with *dates*
- ...

Other notable packages: `readxl` (reading Excel files), `sf` (spatial data), `tidymodels` (modeling & machine learning), `data.table` (big data handling), and many more.

Loading packages

After installation, packages have to be loaded into the environment using `library()`:

```
1 library(tidyverse)
```

Once a package is loaded, you can use functions and/or data inside them:

```
1 data("diamonds") # Places data in your global environment
2 head(diamonds) # Displays first six elements of an object
```

```
# A tibble: 6 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

Recap

- RStudio includes the editor, console, environment, and outputs. R is the underlying engine.
- Objects: the building block of everything in R
  - Vectors
  - Data frames
  - …
- Packages extend what R can do. `tidyverse` is a widely used collection of packages.

Next up

Now that we can orient ourselves in RStudio, create objects, and load packages, it is time to make something with them.

Part 2 - Data visualization with `ggplot2`